
Building a Multilingual GPT-2 from Scratch

A decoder-only transformer with a RoPE vs. learned-positional-embedding ablation, trained on English, Hindi & Marathi

Architecture	Decoder-only, 12 layers, 768-dim, ~117M params
Languages	English · Hindi · Marathi (2 scripts)
Experiment	RoPE vs. learned positional embeddings
Key result	RoPE converges faster; lower ppl on all 3 languages
Platform	Kaggle, single NVIDIA T4

[Yuvraj Rajure]

August 2026 · Inspired by Andrej Karpathy's nanoGPT

1 Summary

This project implements a GPT-2–style, decoder-only language model **entirely from scratch** in PyTorch — tokenizer, self-attention, transformer blocks, training loop, and sampling — following the educational approach of Andrej Karpathy’s nanoGPT, then extends it in three directions the brief flagged as “beyond the basics”:

1. **Multilingual training** on English, Hindi, and Marathi (Latin and Devanagari scripts).
2. **A custom attention modification** — replacing learned absolute positional embeddings with **Rotary Position Embeddings (RoPE)**, run as a controlled A/B experiment.
3. **Full GPT-2-small–class scale** — a 12-layer, 768-dimensional, ~117M-parameter model.

The central experiment trains two otherwise-identical models — one with learned positional embeddings (“baseline”), one with RoPE — for 8,000 steps each on identical data, comparing them by training loss, per-language held-out perplexity, and generation quality.

Headline finding. RoPE converged faster through the middle of training and achieved marginally lower perplexity across *all three* languages, while using *fewer* parameters. Both models reached a similar final training loss. The models are deliberately under-trained (~262M tokens, ~5% of GPT-2’s budget); the goal was hands-on understanding of the architecture, not production fluency.

2 Objective and scope

The objective was a deep, hands-on understanding of transformer internals by building a decoder-only model and training it to model language. Rather than use a library model, every component was implemented directly: a byte-level BPE tokenizer trained on the project’s own corpus; causal self-attention with an optional rotary-embedding path; pre-norm transformer blocks; a weight-tied token embedding / output projection; AdamW with a cosine schedule and linear warmup; and a streaming, packed data pipeline with a checkpoint/resume loop.

The scope was intentionally bounded by the available compute (a single free-tier GPU, ~2-day window). Every scale decision was made to keep a *complete, correct* experiment inside that budget rather than chase an unreachable quality bar.

3 Model architecture

The model uses a standard GPT-2-small configuration.

Hyperparameter	Value
Layers	12
Attention heads	12
Model dimension (d_{model})	768
Head dimension	64
Feed-forward dimension	3,072 (4×)
Context length	512
Vocabulary size	32,000
Parameters (baseline)	~117.2M
Parameters (RoPE)	~116.8M

Weight tying. The input token-embedding matrix and the output projection share weights, following GPT-2 — saving parameters and improving language-modeling performance.

Pre-norm blocks. LayerNorm precedes each attention and MLP sub-layer, with residual connections around each — the GPT-2 arrangement, which trains more stably than post-norm at this depth.

Attention. Scaled dot-product attention with a causal mask. (The T4 lacks the fused Flash-Attention kernel, so the non-fused path is used; a throughput consideration only, not a correctness one.)

Parameters and RoPE. RoPE has *fewer* parameters than the baseline: it replaces the learned positional-embedding table with a fixed, parameter-free rotation — a small efficiency argument in its favour.

4 Tokenization

A **byte-level BPE** tokenizer with a 32,000-token vocabulary was trained on a sample from all three languages. Byte-level BPE suits the multilingual setting: operating on raw bytes, it represents *any* script — Latin and Devanagari alike — without out-of-vocabulary failures, and handles code-mixed text (Hindi/Marathi documents containing English words, common in real web data) natively within one vocabulary. Training on a multilingual sample ensures the learned merges include efficient sub-word units for Devanagari, not just Latin.

5 Data and the multilingual pipeline

Source. Data is streamed from `allenai/c4` (cleaned Common Crawl, successor to the deprecated `mC4`), using the `en`, `hi`, and `mr` splits. Streaming avoids any large download or local storage — documents are tokenized on the fly.

Language filtering. Multilingual web corpora contain mislabeled documents. A lightweight **Unicode-script filter** screens each document: English must be predominantly Latin-script, Hindi/Marathi predominantly Devanagari (U+0900–U+097F). Since Hindi and Marathi share Devanagari, the filter cannot separate them — but the `c4` splits already do, so the filter only rejects off-script noise within each split. This pure-Python approach replaced a statistical language-ID model that proved incompatible with the environment’s NumPy version — a small but instructive trade-off toward a robust, dependency-free solution.

Packing. Filtered documents are tokenized, joined with an end-of-text separator, and packed into 512-token blocks with no padding. The three language streams are interleaved round-robin so every batch sees a language mixture.

6 Training setup

Mixed precision. `float16` with a gradient scaler. (An earlier P100 attempt failed because the installed PyTorch no longer supports that older architecture; the T4 resolved it. `bfloat16` is unsupported on the T4.)

Checkpoint/resume. The loop saves optimizer, scheduler, and model state every 500 steps and resumes automatically. The final experiment (both models, back-to-back) completed in one unattended background run of ~8.5 hours.

7 The experiment: RoPE vs. learned positional embeddings

7.1 Motivation

Self-attention is order-invariant, so positional information must be injected. The two schemes differ in how:

- **Baseline — learned absolute embeddings.** A trainable vector per position is added to token embeddings (original GPT-2).

Setting	Value
Steps per model	8,000
Micro-batch size	16
Gradient accumulation	4 (effective batch = 64)
Tokens per step	32,768
Total tokens per model	~262M
Optimizer	AdamW ($\beta = 0.9, 0.95$; wd 0.1)
Peak learning rate	3×10^{-4}
Schedule	warmup 300 $\rightarrow$ cosine decay
Precision	fp16 with gradient scaling
Gradient clipping	1.0 (global norm)
Hardware	single NVIDIA T4

- **RoPE — rotary embeddings.** Query and key vectors are *rotated* by an angle proportional to position inside attention, so scores depend on the *relative* distance between tokens. RoPE is used in most modern LLMs.

RoPE is a genuine, self-contained attention modification carrying testable advantages: parameter-free, expected to improve length generalization, and often faster to converge.

7.2 Experimental control

The comparison changes **exactly one variable**. Both models share identical data, depth, width, vocabulary, optimizer, schedule, step count, and batch size. Any observed difference is attributable to the positional scheme alone.

8 Results

8.1 Training loss

Both models begin near a loss of **10.5**, matching random-guessing loss over a 32,000-token vocabulary ($\ln 32000 \approx 10.4$) — confirming correct initialization. Loss then drops steeply during warmup.

The informative region is roughly **steps 400–3,500**, where the RoPE curve sits visibly *below* baseline: for the same training, RoPE reached lower loss. This is the clearest signal in the experiment and matches RoPE’s reputation for faster convergence. Beyond ~step 4,000 the curves converge, finishing together near **2.3**.

8.2 Held-out perplexity

Perplexity (lower is better) was measured on freshly streamed, held-out text per language.

Language	Baseline ppl	RoPE ppl	Winner
English (en)	210.3	189.5	RoPE
Hindi (hi)	6.4	6.1	RoPE
Marathi (mr)	6.5	6.3	RoPE

RoPE achieved lower perplexity on all three languages — small but consistent, most pronounced in English (~10% relative). With the faster convergence above, the picture is coherent: the parameter-free scheme was at least as good as, and modestly better than, learned embeddings.

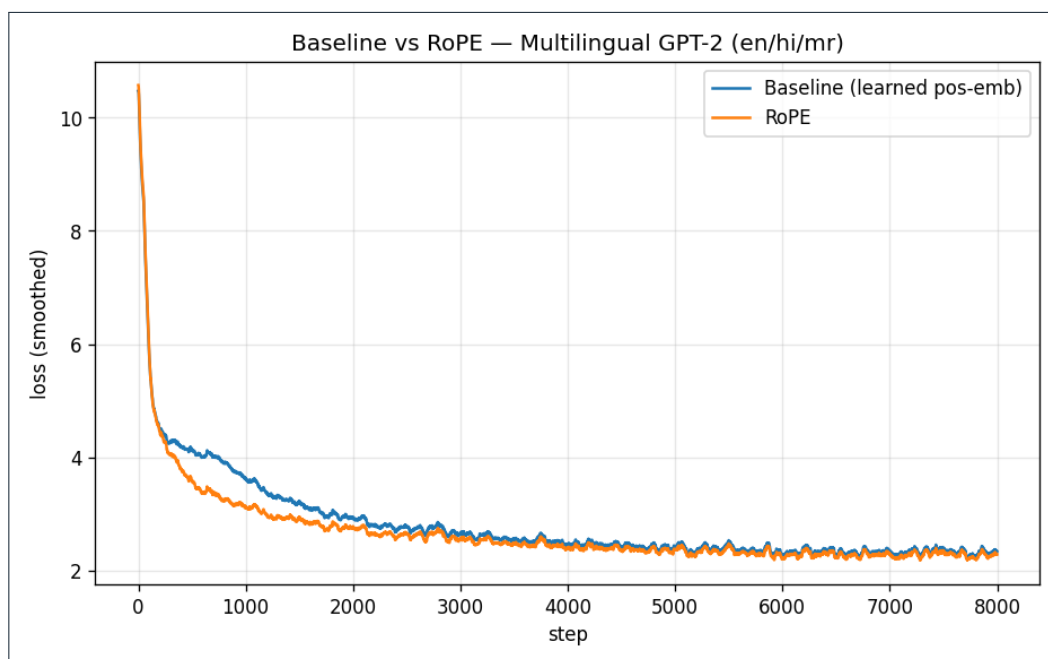


Figure 1: Smoothed training loss, baseline vs. RoPE. RoPE converges faster through the middle of training; both reach a similar final loss.

Interpretation caveat. The gap between English (~ 190 – 210) and Hindi/Marathi (~ 6) is *not* evidence the model “understands” Hindi better. Perplexity is per-token, and the byte-level tokenizer segments each script with different granularity, so cross-language values are not directly comparable. The valid comparison is *within* a language (baseline vs. RoPE) — where RoPE wins consistently.

8.3 Generation quality

Sampling (temperature ≈ 0.7 , top- $k \approx 30$) produces text that is **locally fluent but not globally coherent**: short clauses are grammatical and often topical in the Devanagari languages, but over longer spans the text drifts and occasionally loops on high-frequency phrases.

This is expected and analyzed, not hidden: at ~ 262 M tokens ($\sim 5\%$ of GPT-2’s budget) the model learns token- and phrase-level statistics but not long-range discourse structure. It is a **base language model** — it continues text — not an instruction-tuned assistant; it does not answer questions, as that would require a separate fine-tuning stage. Presenting it as a “text generator” rather than a “chatbot” is the accurate framing.

9 Discussion: scale and limitations

The dominant limitation is the **token budget**, not the architecture. Reference points: GPT-2 (124M) trained on ~ 8 billion tokens; the compute-optimal (“Chinchilla”) guideline of ~ 20 tokens/parameter implies ~ 2.2 billion tokens for a 117M model; this project used ~ 262 M per model — roughly **12% of the compute-optimal minimum** and **3% of GPT-2’s actual budget**.

Additional compute would most productively go to (1) more training tokens, the biggest lever on coherence; (2) fewer languages, raising per-language token counts; (3) longer context. None change the architecture — they change how much it gets to learn. Further honest limitations: “held-out” perplexity is approximate because data is streamed rather than split from a fixed disjoint set, and per-language quality tracks each language’s representation and tokenization efficiency.

10 How the project addresses the brief

Brief requirement	How it was met
Decoder-only model, from scratch	Hand-implemented attention, MLP, blocks, tokenizer, loop — no library model
Tokenization	Own byte-level BPE, 32k vocab, project corpus
Self-attention & sequence modeling	Causal attention; packed 512-token blocks; next-token objective
Train to generate language	Two models trained; multilingual generation shown
<i>Bonus:</i> multilingual data	English + Hindi + Marathi; two scripts; script filtering
<i>Bonus:</i> custom attention	RoPE vs. learned-positional controlled ablation
<i>Bonus:</i> full GPT-2 config	12-layer / 768-dim / ~117M GPT-2-small-class

The project delivers the required core plus all three suggested bonus directions, with a controlled experiment producing a concrete, defensible result at its centre.

11 Conclusion and future work

A GPT-2-small-class decoder-only transformer was built from scratch and trained on three languages; a controlled RoPE-vs-baseline ablation showed RoPE converging faster and giving a small, consistent perplexity improvement across all three languages while using fewer parameters. The models are undertrained by design, and their locally-fluent, globally-drifting output is what theory predicts at this scale — understanding *why* was a primary learning outcome.

Next steps: train substantially longer (toward the ~2B-token compute-optimal point); test RoPE's expected **length-generalization** advantage at context lengths beyond the 512 seen in training; add a fixed held-out validation set; and, as a separate stage, instruction-fine-tune the base model for question-answering behaviour.

12 Reproducibility

Artifacts: `ckpt_baseline/last.pt`, `ckpt_rope/last.pt`; `tokenizer/vocab.json`, `tokenizer/merges.txt`; `per-model losses.json`; `comparison.png`; `perplexity_results.json`.

To reproduce: run the notebook top to bottom with `use_rope=False` (baseline) and `use_rope=True` (RoPE), or the combined cell that trains both. One ~8.5-hour background run on a T4 reproduces both models and all evaluation artifacts.

Interactive demo: a Streamlit app loads either model (or both, side by side), generates text from a prompt in any of the three languages, and shows the perplexity table and loss curve.